

SmolVLM

Redefining Small and Efficient Multimodal Models

A compact VLM family from Hugging Face and Stanford that targets competitive multimodal performance with dramatically lower memory footprints.

256M

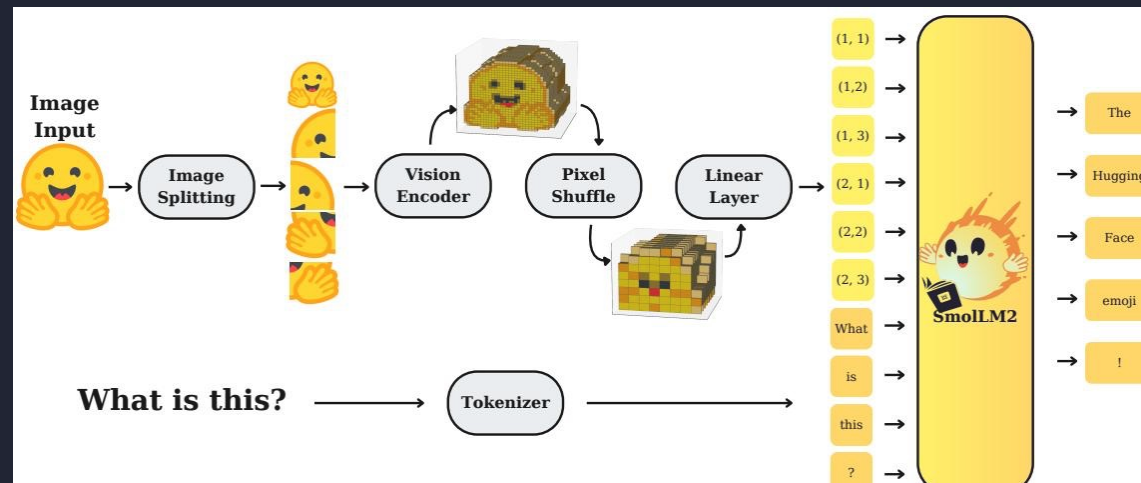
smallest model

0.8

GB RAM, b=1

16K

max context



Authors: Hugging Face & Stanford University

Large VLMs Are Powerful but Impractical for Edge Deployment

State-of-the-art capability collides with memory budgets on mobile and edge devices.

Deployment bottleneck

- Large open-source VLMs require high GPU memory, limiting mobile and edge use.
- On-device inference needs compact checkpoints, low activation memory, and fast decoding.
- The design target shifts from raw scale to architecture-level efficiency.

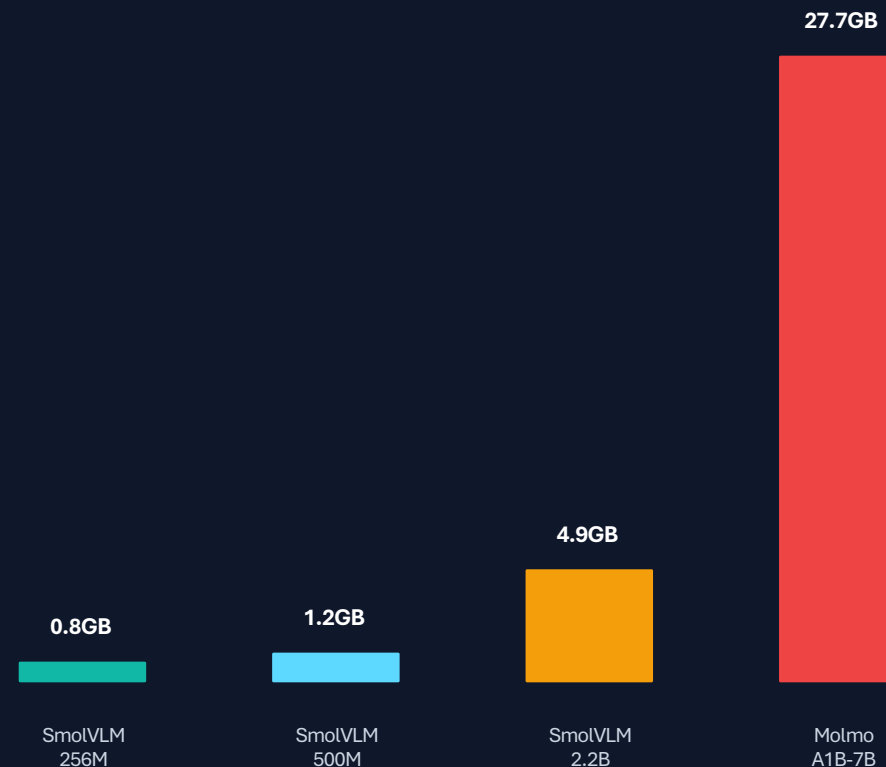
27.7

GB RAM for Molmo-A1B-7B

0.8

GB RAM for SmolVLM-256M

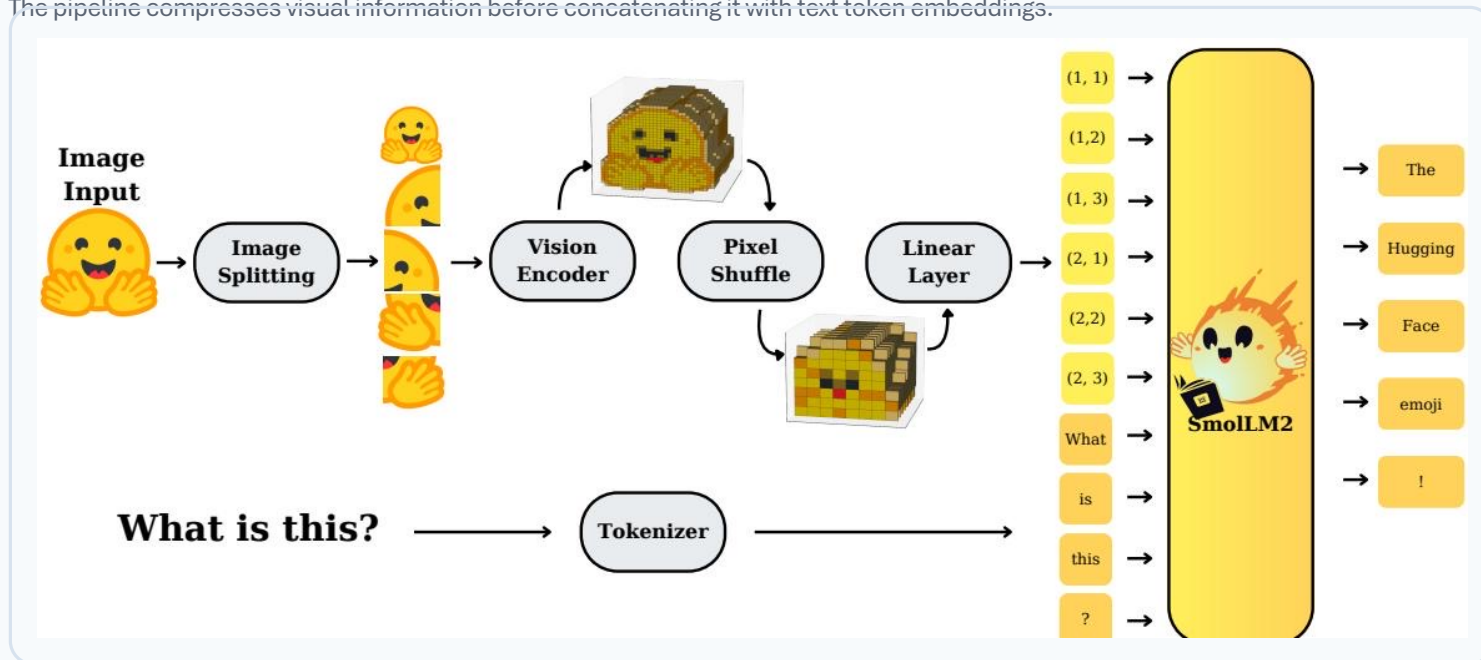
Memory footprint comparison



SmolVLM frames compact multimodal modeling as a compute-allocation problem, not only a scaling problem.

SmolVLM Architecture: Image Splitting, Pixel Shuffle, and SmolLM2 Backbone

The pipeline compresses visual information before concatenating it with text token embeddings.



REFERENCE: ARCHITECTURE FIGURE

Core idea: preserve enough visual signal for reasoning while sharply reducing the sequence length seen by the language model.

1

Split inputs

Images are split into sub-images; video frames are sampled and processed similarly.

2

Encode vision

A SigLIP vision encoder transforms image patches into visual features.

3

Compress tokens

Pixel shuffle performs space-to-depth compression before projection.

4

Fuse with language

A linear layer maps features into SmolLM2 input space and concatenates with text.

Smaller Vision Encoders Outperform Larger Ones in Compact VLMs

Finding 1: balanced encoder-LM compute matters more than maximizing the vision tower.

Compact LM pairing: 135M language model

SigLIP-B/16

outperforms

SigLIP-SO400M

larger encoder

428M param

Qualitative experimental result: with the smallest LMs, the 93M SigLIP-B/16 encoder beats the 428M SigLIP-SO400M despite using far fewer vision parameters.

Allocation principle

1

Small LM regime

Use a compact encoder; avoid starving the language model.

2

Larger LM regime

At 1.7B LM scale, the larger encoder can justify its 10% parameter overhead.

3

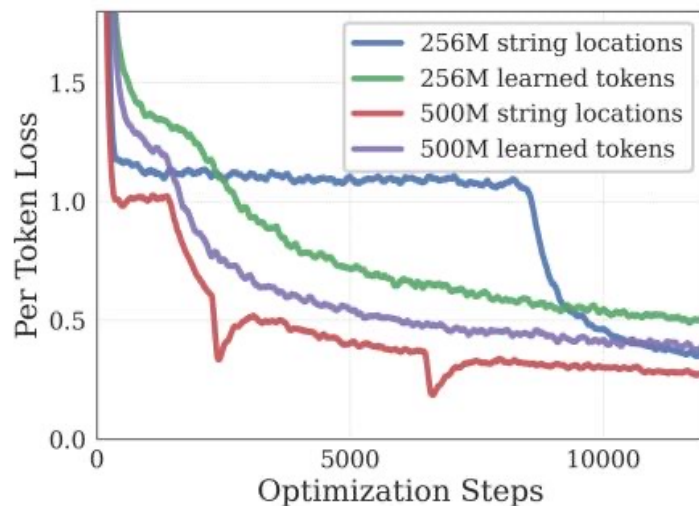
Design implication

Match encoder capacity to LM scale instead of optimizing components independently.

Aggressive Pixel Shuffle (r=4) Cuts Visual Tokens 16× for Small Models

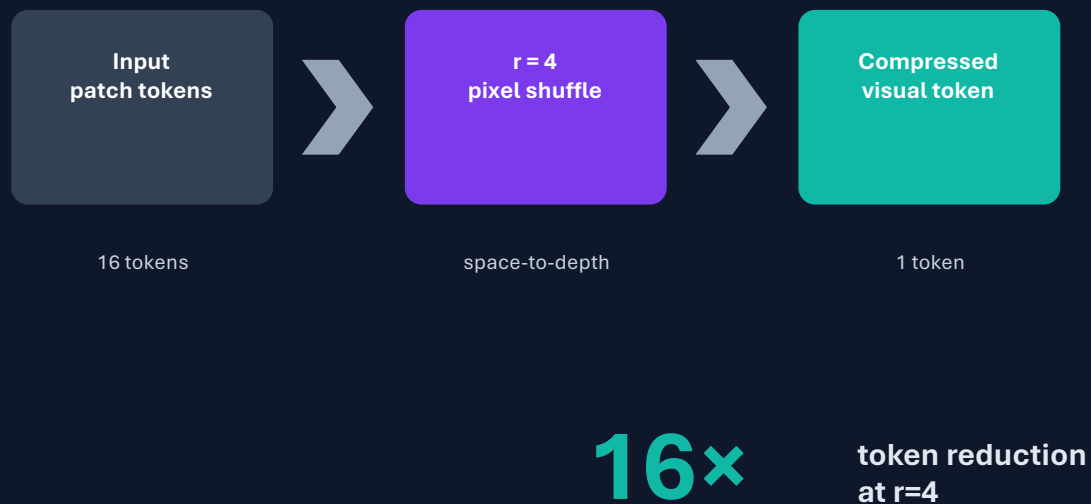
Finding 3: small VLMs benefit from stronger visual-token compression; larger models use r=2.

Pixel shuffle: space → depth



Visual tokens are reduced by r^2 while spatial features are rearranged into channels.

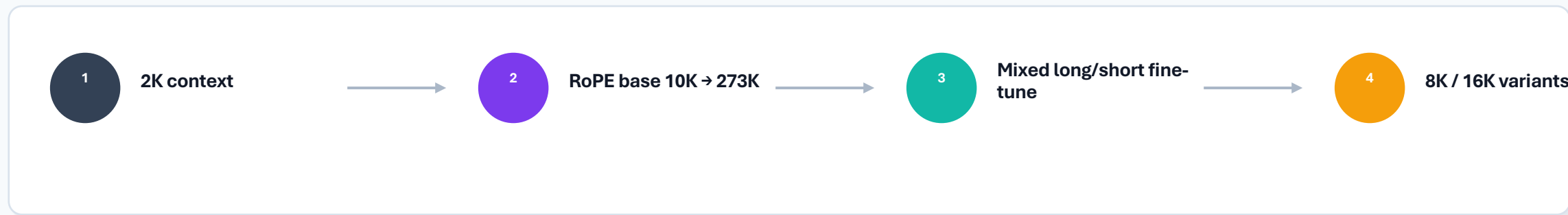
Compression factor



Empirical guidance: small VLMs benefit from r=4 because the lower attention overhead improves long-context modeling; larger models prefer r=2.

Extending Context from 2K to 16K Tokens Unlocks Higher Image Resolution

Finding 2: longer context improves performance and enables more visual tokens from high-resolution images.



16K

context for SmolVLM-2.2B

The highest-capacity variant uses extended context to support higher-resolution visual inputs.

8K

context for 256M / 500M

Smaller variants still receive a 4× extension over the 2K baseline while preserving compact deployment.

273K

RoPE base after adjustment

Long-context adaptation is paired with mixed long/short-context data for stable fine-tuning.

Design consequence: context length becomes a first-class capacity knob for compact multimodal models.

SmolVLM Achieves Competitive Performance at Sub-1GB to 4.9GB RAM

Benchmark comparison across single-image, multi-task, and video tasks.

Capability	Benchmark	256M	500M	2.2B	Best Efficient OS
Single-Image	OCRBench	52.6%	61.0%	72.9%	54.7% Molmo-A1B-7B
Single-Image	AI2D	46.4%	59.2%	70.0%	71.0% Molmo-A1B-7B
Single-Image	ChartQA	55.6%	62.8%	68.7%	48.0% Molmo-A1B-7B
Single-Image	TextVQA	50.2%	60.2%	73.0%	61.5% Molmo-A1B-7B
Single-Image	DocVQA	58.3%	70.5%	80.0%	77.7% Molmo-A1B-7B
Single-Image	ScienceQA	73.8%	80.0%	89.6%	87.5% Molmo-A1B-7B
Multi-task	MMMU	29.0%	33.7%	42.0%	33.9% Molmo-A1B-7B
Multi-task	MathVista	35.9%	40.1%	51.5%	37.6% Molmo-A1B-7B
Multi-task	MMStar	34.6%	38.3%	46.0%	43.1% Molmo-A1B-7B
Video	Video-MME	33.7%	42.2%	52.1%	45.0% InternVL2-2B
Video	MLVU	40.6%	47.3%	55.2%	48.2% InternVL2-2B
Average	All Benchmarks	44.0%	51.0%	59.8%	—

RAM usage (batch=1)

256M 0.8GB

500M 1.2GB

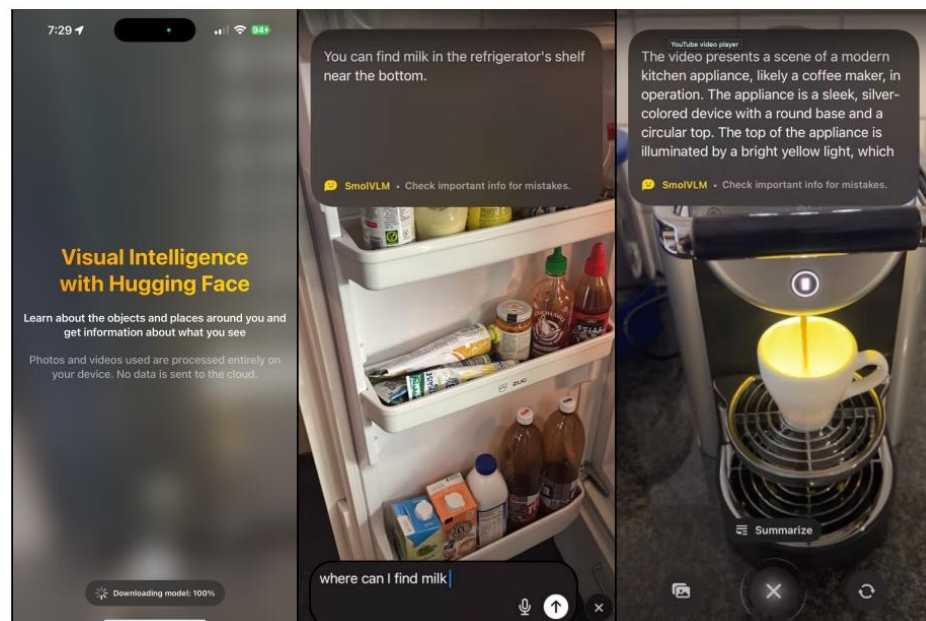
2.2B 4.9GB

Molmo-A1B-7B 17.7GB

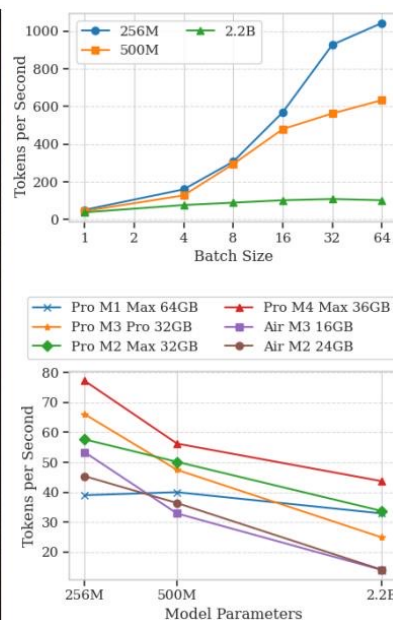
Key paper finding: SmolVLM-256M surpasses the 300× larger Idefics-80B.

Real-Time Inference on iPhones: Up to 80 Tokens/Second on Apple Silicon

HuggingSnap runs locally with no cloud dependency; throughput scales strongly for the 256M model.



REFERENCE: MOBILE APP + THROUGHPUT FIGURE



On-device metrics

~80

tokens/s

SmolVLM-256M on Pro M1 Max 64GB

~55

tokens/s

256M on MacBook Air M3 16GB

~1000

tokens/s

batch size 64 on high-end devices

Runs photos and videos entirely locally, enabling privacy-preserving edge multimodal applications.

Operational takeaway: a sub-1GB VLM can deliver interactive decoding rates on consumer Apple Silicon.

Key Takeaways

Design principles for efficient multimodal models

1

Balance encoder-LM compute

Small LMs can perform better with a smaller 93M SigLIP-B/16 encoder than a 428M vision tower.

2

Treat context as capacity

Extend context from 2K toward 8K–16K with RoPE-base adjustment and mixed-context fine-tuning.

3

Compress visual tokens aggressively

Use $r=4$ pixel shuffle for small models to reduce visual tokens 16 $\times$ and lower attention overhead.

4

Optimize for memory targets

Sub-1GB inference is achievable: SmolVLM-256M uses 0.8GB GPU RAM at batch size 1.

5

Open the deployment loop

Weights, datasets, code, and HuggingSnap are open-sourced for reproducible on-device AI research.

Efficient VLMs are an architecture problem

SmolVLM shows that careful compute allocation, token compression, and context scaling can make multimodal reasoning practical on edge devices.

competitive performance • low RAM • local inference